MODULE 1

Introduction to Data Warehouse

1.1 Data warehouse: Basic concepts

A data warehouse is a repository of information collected from multiple sources, stored under a unified schema, and usually residing at a single site.

"A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process."

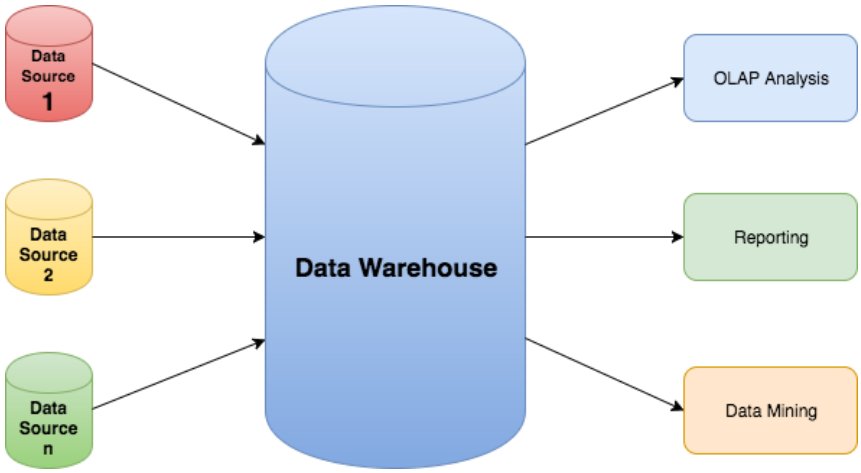
Following are the features of data warehouse:

Subject-Oriented: A data warehouse can be used to analyse a particular subject area. For example, "sales" can be a particular subject.

Integrated: A data warehouse integrates data from multiple data sources. For example, source A and source B may have different ways of identifying a product, but in a data warehouse, there will be only a single way of identifying a product.

Time-Variant: Historical data is kept in a data warehouse. For example, one can retrieve data from 3 months, 6 months, 12 months, or even older data from a data warehouse. This contrasts with a transactions system, where often only the most recent data is kept.

Non-volatile: Once data is in the data warehouse, it will not change. So, historical data in a data warehouse should never be altered.



1.2 Data warehousing

Data warehousing is the process of constructing and using data warehouses.

The construction of a data warehouse requires:

Data cleaning

Data integration

Data consolidation.

1.3 Differences between Operational Database Systems and Data Warehouses

- Online operational database systems are to perform online transaction and query processing. These systems are called **online transaction processing** (OLTP) systems.
- Data warehouse systems, serve users in the role of data analysis and decision making. Such systems can organize and present data in various formats. These systems are known as **online analytical processing** (OLAP) systems.

Comparison of OLTP and OLAP:

Features	**OLTP**	**OLAP**
Characteristic	operational processing	informational processing
Orientation	Transaction	Analysis
User	Clients, clerks, IT professionals	knowledge worker (e.g., manager executive, analyst)
DB design	entity-relationship (ER) model	star/snowflake
Data	Current data	historic
Summarization	primitive, highly detailed	summarized, consolidated
Unit of work	short, simple transaction	complex query
Access	read/write	mostly read
Focus	data in	information out

Number of records Accessed	Tens	Millions
Number of users	Thousands	Hundreds
DB size	GB to high-order GB	>=TB

1.4 Characteristics of OLAP

Users and system orientation:

An OLTP system is customer-oriented

An OLAP system is market-oriented

Data contents

An OLTP system manages current data

An OLAP system manages large amounts of historic data

Database design

An OLTP system adopts an entity-relationship (ER) data model and an application-oriented database design

An OLAP system typically adopts either a star or a snowflake model and a subject-oriented database design.

View

An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historic data

OLAP systems also deal with information that originates from different organizations, integrating information from many data stores.

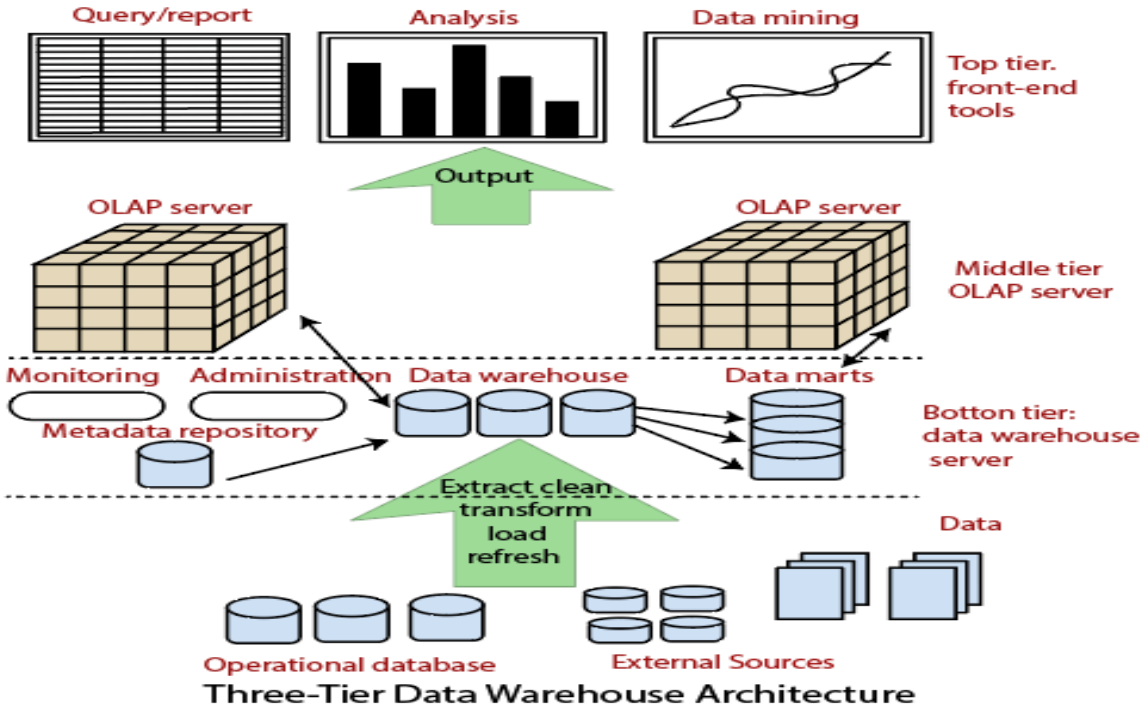
Access patterns

Access patterns of an OLTP system consist mainly of short, atomic transactions

Accesses to OLAP systems are mostly read-only operations

1.5 Data Warehousing: A Multitier Architecture

Data warehouses often adopt a three-tier architecture, as presented in Figure below



Three-Tier Data Warehouse Architecture

1. The bottom tier is a warehouse database server that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (e.g., customer profile information provided by external consultants).

2. The middle tier is an OLAP server that is typically implemented using either

(1) A relational OLAP(ROLAP) model (i.e., an extended relational DBMS that maps operations on multidimensional data to standard relational operations); or

(2) A multidimensional OLAP (MOLAP) model (i.e., a special purpose server that directly implements multidimensional data and operations).

3. The top tier is a front-end client layer, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

Extraction, Transformation, and Loading

Data warehouse systems use back-end tools and utilities to populate and refresh their data.

These tools and utilities include the following functions:

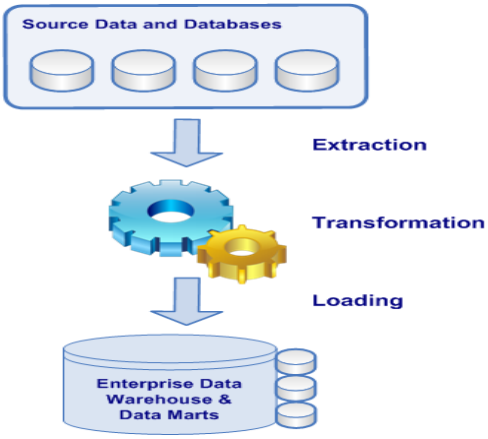
Data extraction

Data cleaning

Data transformation

Load

Refresh



Data extraction: Gathers data from multiple, heterogeneous, and external sources.

Data cleaning: Detects errors in the data and rectifies them when possible.

Data transformation: Converts data from legacy or host format to warehouse format.

Load: Sorts, summarizes, consolidates, computes, checks integrity, and imports the data into data warehouse

Refresh: Propagates the updates from the data sources to the warehouse.

1.6 Data Warehouse Models

From the architecture point of view, there are three data warehouse models

Enterprise Warehouse

Data Mart

Virtual Warehouse

→ Enterprise warehouse

An enterprise warehouse collects all of the information about subjects spanning the entire organization.

It provides corporate-wide data integration, usually from one or more operational systems or external information providers, and is cross-functional in scope.

It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond.

An enterprise data warehouse may be implemented on traditional mainframes, computer super servers, or parallel architecture platforms.

It requires extensive business modelling and may take years to design and build.

→ Data mart

A data mart contains a subset of corporate-wide data that is of value to a specific group of users. The scope is confined to specific selected subjects.

For example, a marketing data mart may confine its subjects to customer, item, and sales.

The data contained in data marts tend to be summarized.

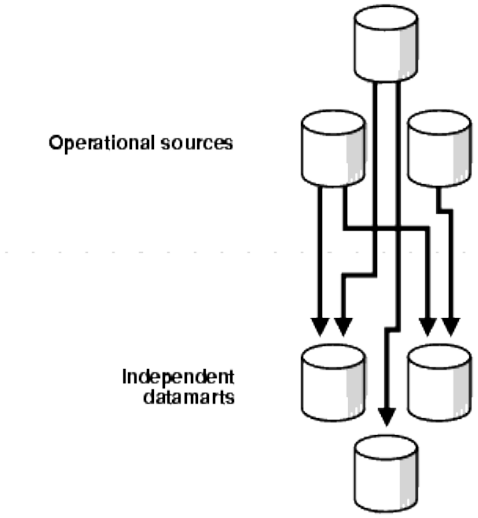
Data marts are usually implemented on low-cost departmental servers that are Unix/Linux or Windows based

Depending on the source of data, data marts can be categorized as

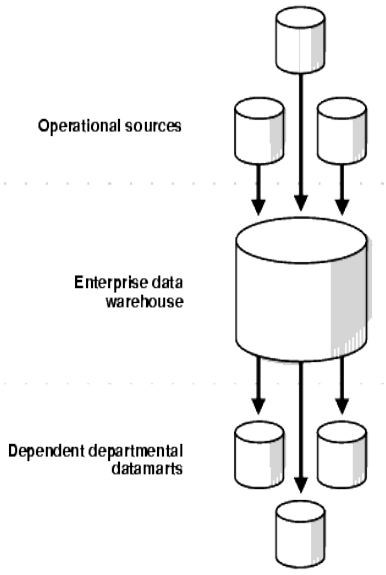
Independent data marts

Dependent data marts

Independent data marts are sourced from data captured from one or more operational systems or external information providers, or from data generated locally within a particular department or geographic area.



Dependent data marts are sourced directly from enterprise data warehouses



→ Virtual warehouse

A virtual warehouse is a set of views over operational databases.

For efficient query processing, only some of the possible summary views may be materialized.

A virtual warehouse is easy to build but requires excess capacity on operational database servers

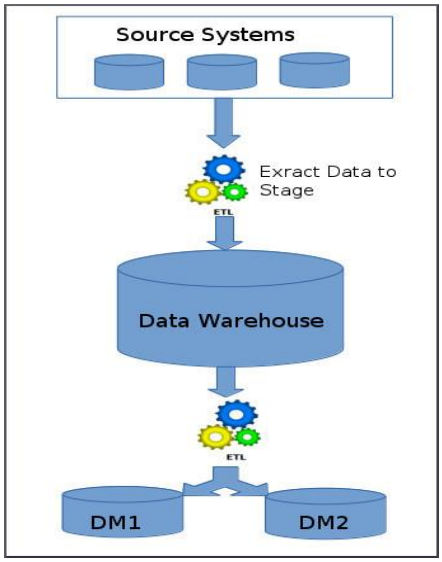
1.7 Data Warehouse Design Approaches

Top-down Data Warehouse Design Approach

In the top-down approach, the data warehouse is designed first and then data mart are built on top of data warehouse.

Steps that are involved in top-down approach:

- Data is extracted from the various source systems. The extracts are loaded and validated in the stage area. Validation is required to make sure the extracted data is accurate and correct. Use the ETL tools or approach to extract and push to the data warehouse.
- Data is extracted from the data warehouse. Various aggregation, summarization techniques are used to extract data and loaded back to the data warehouse.
- Once the aggregation and summarization is completed, various data marts extract that data and apply the some more transformation to make the data structure as defined by the data marts.



→It is Expensive

→It is inflexible to changing departmental changes

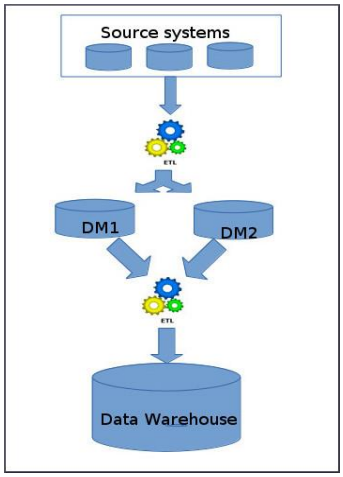
→Takes a long time to develop

Bottom-up Data Warehouse Design Approach

In this Approach, Data marts are directly loaded with the data from the source systems and then ETL process is used to load in to Data Warehouse.

Steps that are involved in bottom-up approach:

- The data flow in the bottom up approach starts from extraction of data from various source system into the stage area where it is processed and loaded into the data marts that are handling specific business process.
- After data marts are refreshed the current data is once again extracted in stage area and transformations are applied to create data into the data mart structure. The data is the extracted from Data Mart to the staging area is aggregated, summarized and so on loaded into EDW



→Provides flexibility

→low cost

→ Problem is when integrating various disparate data marts into a consistent enterprise data warehouse.

A recommended method for the development of data warehouse systems is to implement the warehouse in an **incremental and evolutionary** manner

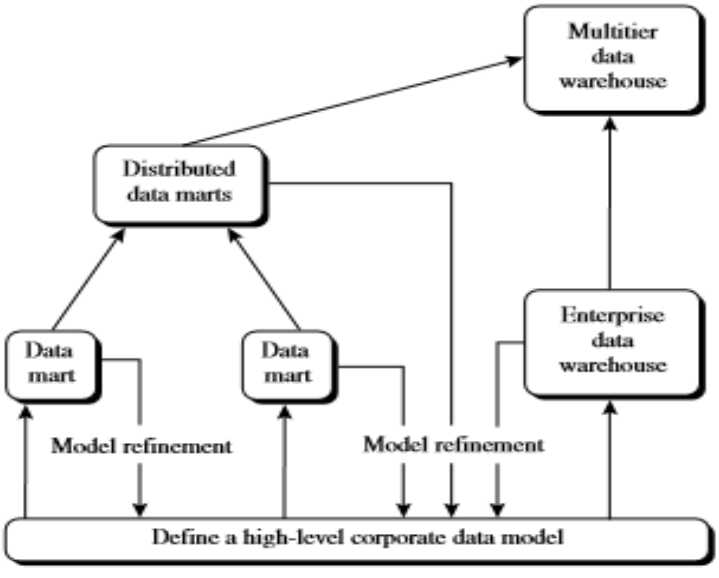


Figure 4.2 A recommended approach for data warehouse development.

First, a high-level corporate data model is defined within a reasonably short period (such as one or two months) that provides a corporate-wide, consistent, integrated view of data among different subjects and potential usages.

Second, independent data marts can be implemented in parallel with the enterprise warehouse based on the same corporate data model set noted before.

Third, distributed data marts can be constructed to integrate different data marts via hub servers.

Finally, a multitier data warehouse is constructed where the enterprise warehouse is the sole custodian of all warehouse data, which is then distributed to the various dependent data marts.

1.9 Data Cube and OLAP

A **data cube** allows data to be modelled and viewed in multiple dimensions.

It is defined by

dimensions

facts

Dimensions are the entities with respect to which an organization wants to keep records

For example:

AllElectronics may create a sales data warehouse in order to keep records of the store's sales with respect to the dimension's time, item, branch, and location.

Facts are numeric measures.

Examples of facts for a sales data warehouse include ***dollars sold*** (sales amount in dollars), ***units sold*** (number of units sold)

2-D Data cube

Table 1.1: 2-D View of Sales Data for *All Electronics* According to dimensions time and item, where sales are from branches located in the city Vancouver.

location = "Vancouver "				
	item (type)			
Time (quarter)	**Home entertainment**	**Computer**	**Phone**	**Security**
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

3-D Data cube (series of 2-D)

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
	item				item				item				item			
	home				home				home				home			
time	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Data for *All Electronics* According to *time*, *item*, and *location*

3-D Data cube

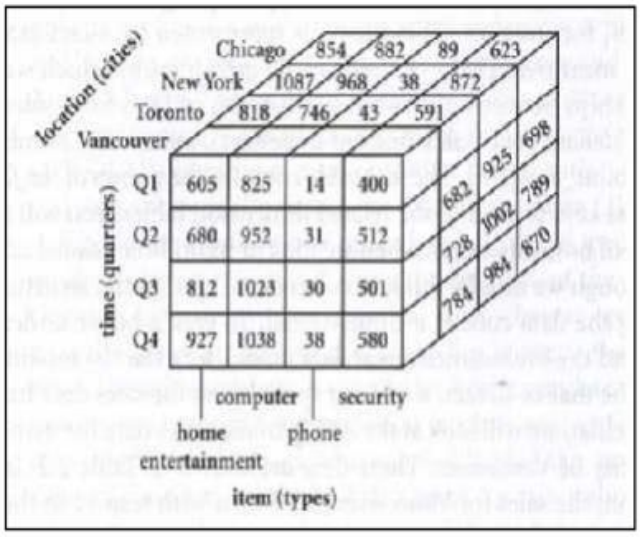


Figure 1.3: 3-D Data Cube

4-D Data Cube (series of 3-D)

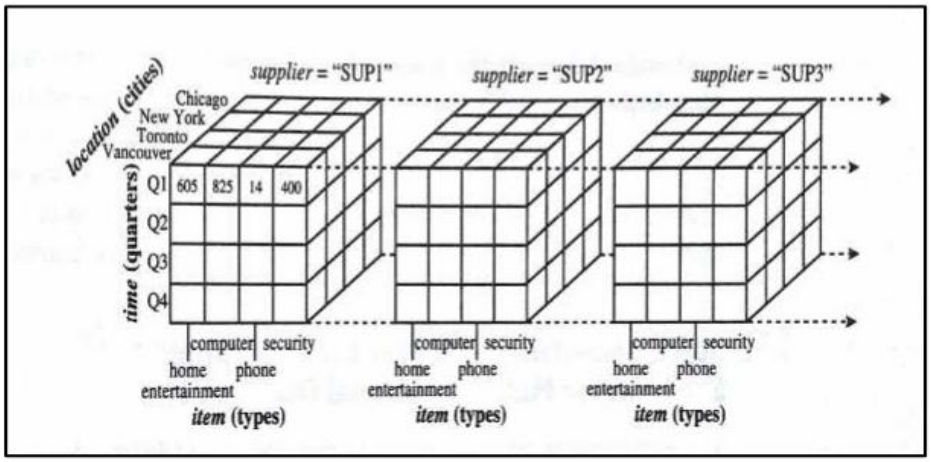


Figure 1.4: 4-D data cube representation of sales data, according to *time*, *item*, *location*, and *supplier*.

Multidimensional data model can be summarised as:

Given a set of dimensions, we can generate a **cuboid** for each of the possible subsets of the given dimensions. The result would form a **lattice of cuboids**

The 0-D cuboid, which holds the highest level of summarization, is called the apex cuboid

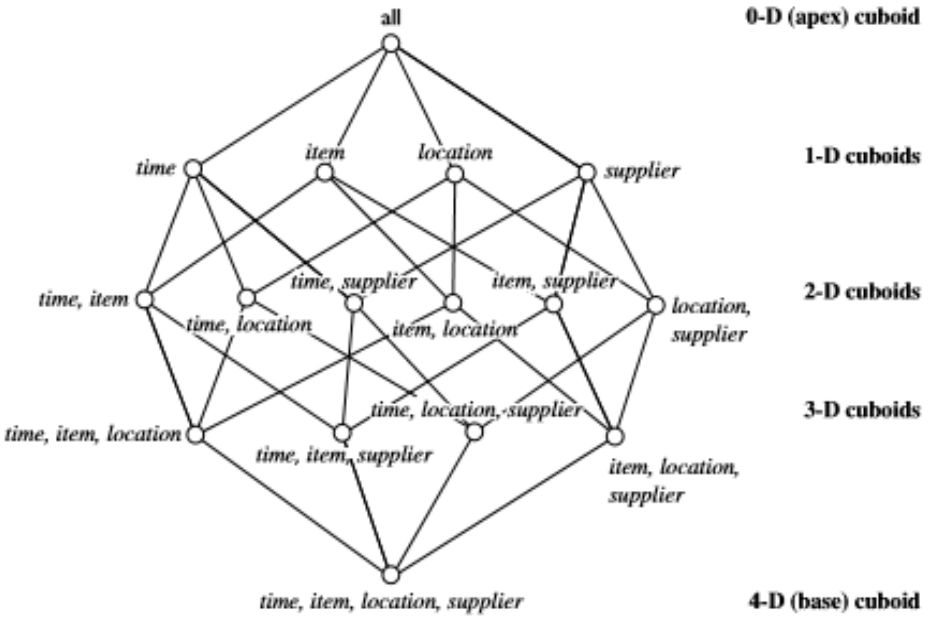


Figure 4.5 Lattice of cuboids, making up a 4-D data cube for *time*, *item*, *location*, and *supplier*. Each cuboid represents a different degree of summarization.

1.10 Schemas for Multidimensional Data Model

Data model for a data warehouse is a multidimensional model, which can exist in the form of:

Star schema

Snowflake schema

Fact constellation schema

Star schema: The most common modeling paradigm is the star schema, in which the data warehouse contains (1) a large central table (fact table) containing the bulk other data, with no redundancy, and (2) a set of smaller attendant tables (dimension tables), one for each dimension. The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table.

Example:

1.10.1 Star schema.

A star schema for All Electronics sales is shown in Figure

Sales are considered along four dimensions: time, item, branch, and location. The schema contains a central fact table for sales that contains keys to each of the four dimensions, along with two measures: dollars sold and units sold. To minimize the size of the fact table, dimension identifiers (e.g., time key and item key) are system-generated identifiers.

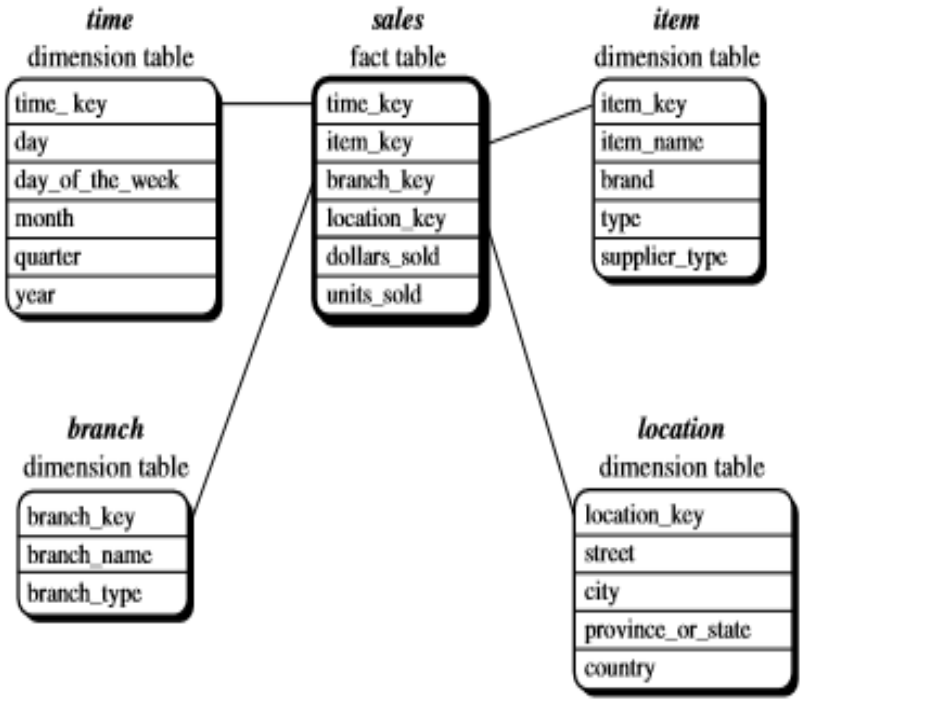


Figure 3.4 Star schema of a data warehouse for sales.

1.10.2 Snowflake schema:

The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, thereby further splitting the data into additional tables.

The resulting schema graph forms a shape similar to a snowflake.

The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies.

Such a table is easy to maintain and saves storage space. However, this space savings is negligible in comparison to the typical magnitude of the fact table.

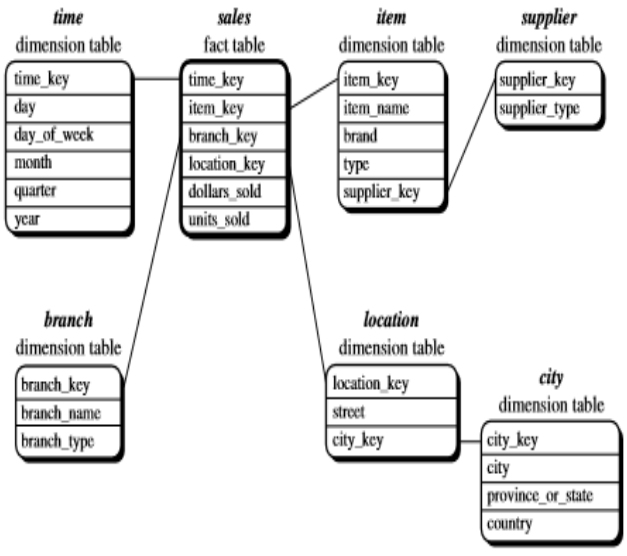


Figure 3.5 Snowflake schema of a data warehouse for sales.

The main difference between the two schemas is in the definition of dimension tables. The single dimension table for item in the star schema is normalized in the snowflake schema, resulting in new item and supplier tables. For example, the item dimension table now contains the attributes item key, item name, brand, type, and supplier key, where supplier key is linked to the supplier dimension table, containing supplier key and supplier type information. Similarly, the single dimension table for location in the star schema can be normalized into two new tables: location and city. The city key in the new location table links to the city dimension. Notice that, when desirable, further normalization can be performed on province or state and country in the snowflake schema.

1.10.3 Fact constellation:

Sophisticated applications may require multiple fact tables to share dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.

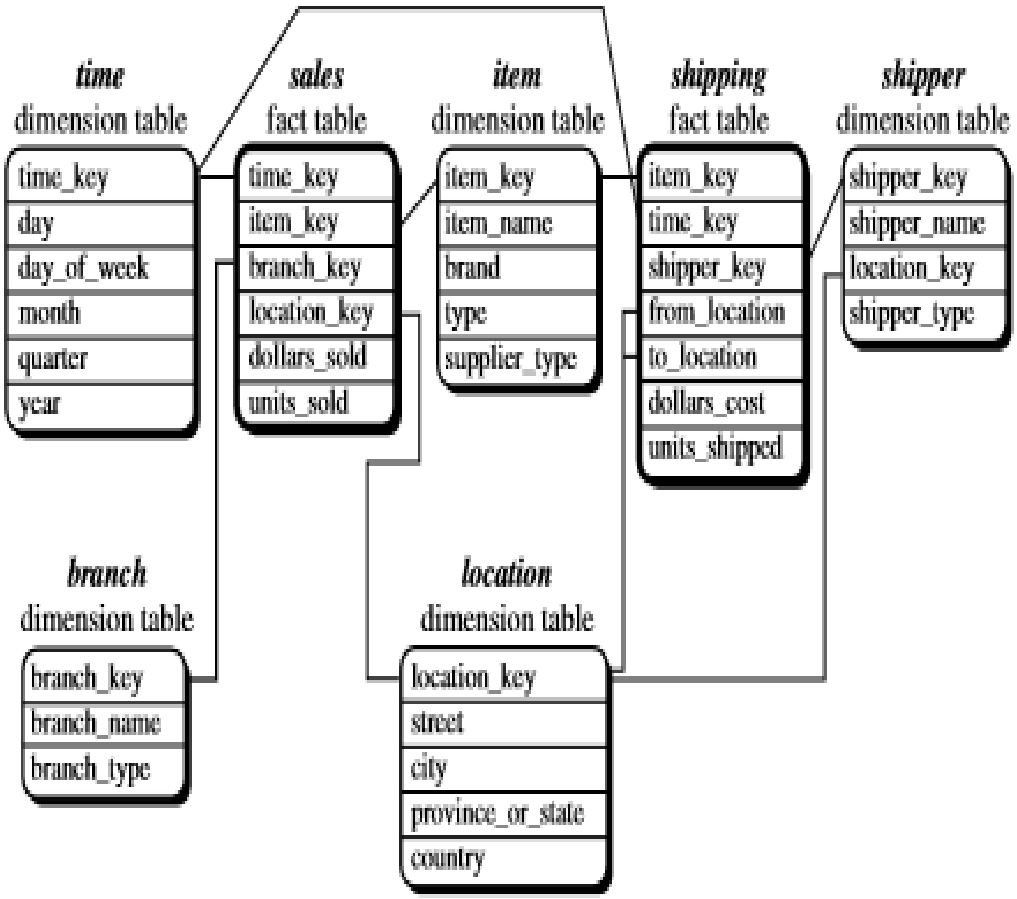


Figure 3.6 Fact constellation schema of a data warehouse for sales and shipping.

This schema specifies two fact tables, sales and shipping.

The sales table definition is identical to that of the star schema.

The shipping table has five dimensions, or keys—item key, time key, shipper key, from location, and to location— and two measures—dollars cost and units shipped.

A fact constellation schema allows dimension tables to be shared between fact tables.

For example, the dimension tables for time, item, and location are shared between the sales and shipping fact tables.

1.10.4 Difference between star and snowflakes

Comparison	Star schema	Snowflake schema
Structure of schema	Contains fact and dimension tables.	Contains sub-dimension tables including fact and dimension tables.
Use of normalization	Doesn't use normalization.	Uses normalization
Ease of use	Simple to understand and easily designed.	Hard to understand and design.
Space usage	More	Less
Foreign key used	Fewer	Large in number

1.11 Role of concept hierarchy

A **concept hierarchy** defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts.

Consider a concept hierarchy for the dimension location.

City values for location include Vancouver, Toronto, New York, and Chicago.

Each city can be mapped to the province or state to which it belongs. For example, Vancouver can be mapped to British Columbia, and Chicago to Illinois.

The states can in turn be mapped to the country to which they belong, such as Canada or the USA.

These mapping forms a concept hierarchy for the dimension location, mapping set of low level concepts(cities) to more general concepts(countries).

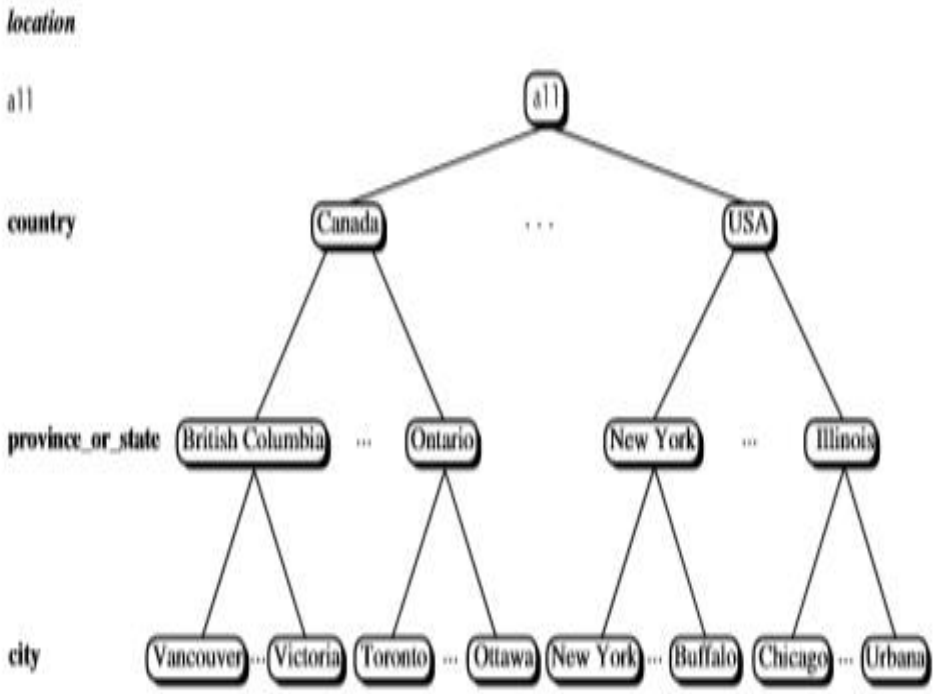


Figure 3.7 A concept hierarchy for the dimension *location*. Due to space limitations, not all of the nodes of the hierarchy are shown (as indicated by the use of "ellipsis" between nodes).

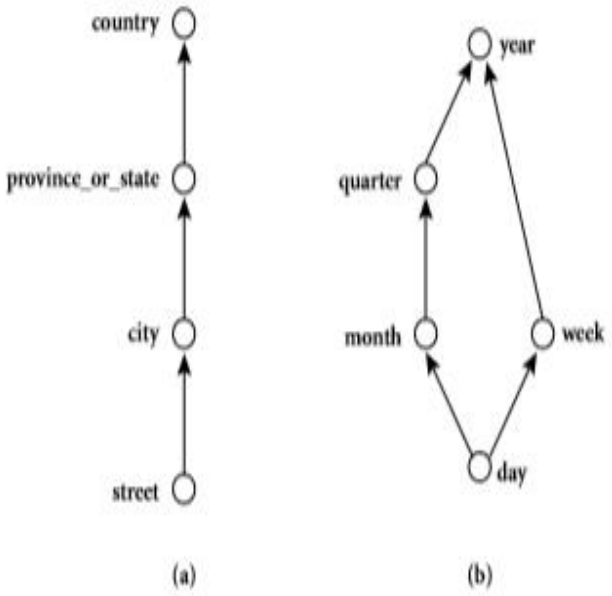


Figure 3.8 Hierarchical and lattice structures of attributes in warehouse dimensions: (a) a hierarchy for *location*; (b) a lattice for *time*.

In the above figure, "street<city<province_or_state<country".

Partial order for the time dimension based on the attributes day, week, month, quarter, and year is "day< {month<quarter; week}< year"

Concept hierarchy can also be defined in a **set-grouping hierarchy.**

Set-grouping hierarchy is shown below for the dimension price, where an interval ($X…$Y] denotes the range from $X(exclusive) to $Y(inclusive).

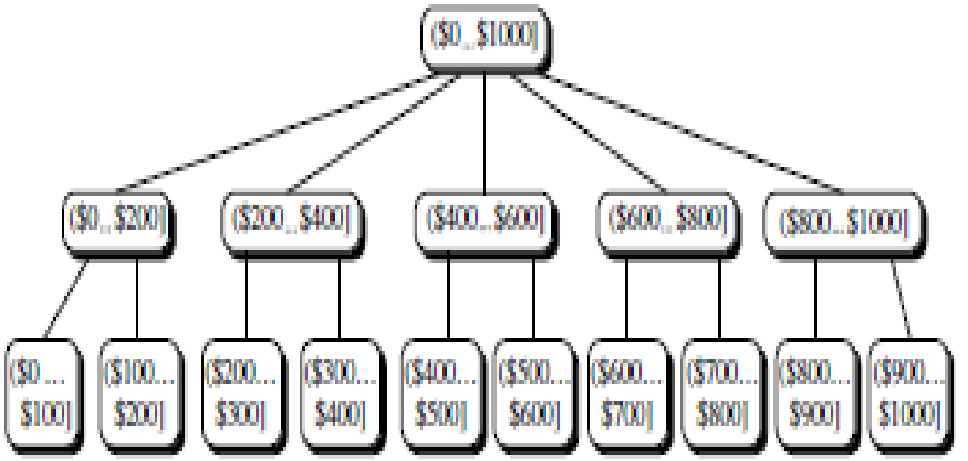


Figure 4.11 A concept hierarchy for *price*.

1.12 Measures: Their Categorization and computation

Measures can be organized into 3 categories based on the Aggregate functions used:

Distributive

Algebraic

Holistic

Distributive Aggregate function

An aggregate function is distributed, if it can be computed in a distributed manner

Suppose the data is partitioned into n sets.

We apply the function to each partition, resulting in n Aggregate values

If the result derived by applying the function to the n aggregate values is the same as that derived by applying the function to entire data set in distributive manner.

For Example, sum() can be computed for a data cube by first partitioning the cube into set of subcubes, computing sum() for each subcube, and then summing up the counts obtained for each subcube. Hence sum() is aggregate function.

Some good examples of distributive aggregate functions are:

count()

min()

max()

sum()

Algebraic Aggregate function

An aggregate function is algebraic, if it can be computed by an algebraic function M arguments(where M is bounded positive integer), each of which is obtained by applying a distributive aggregate function.

A measure is algebraic if it is obtained by applying an algebraic aggregate function

Example:

avg() can be computed by sum()/count()

where both sum() and count() are distributive aggregate function

Holistic Aggregate function

An aggregate function is holistic if there is no constant bound on storage size to describe a sub aggregate.

There does not exist any algebraic function with M arguments(where M is a constant) that characterizes the computation.

Example:

median()

mode()

rank()

1.13 OLAP operation

- Roll-up
- Drill-down
- Slice and dice
- Pivot (rotate)

Roll-up:

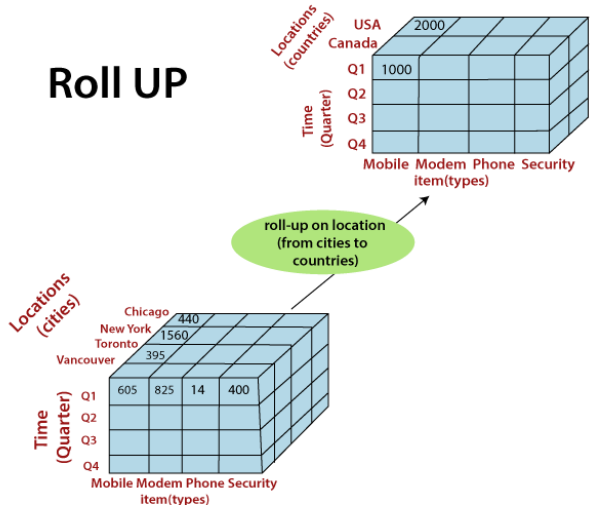
→Also called drill-up operation

→Performs aggregation on a data cube, either by ***climbing up a concept hierarchy or by dimension reduction***

→It navigates from more detailed data to less detailed data.

→Initially the concept hierarchy was "**street < city < state< country**".

→ On rolling up, the data is aggregated by ascending the location hierarchy from the level of city to the level of country.



Drill-down:

→Drill-down is the reverse of roll-up.

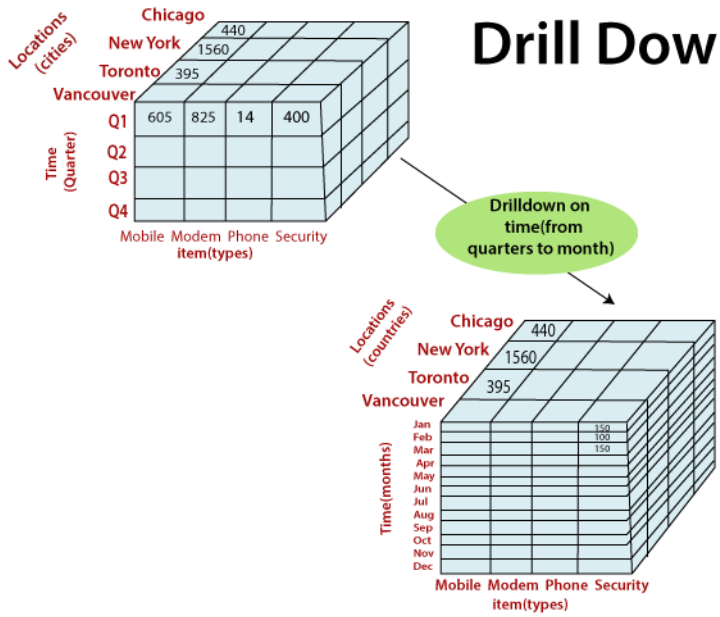
→It navigates from less detailed data to more detailed data.

→Drill-down can be realized by either ***stepping down a concept hierarchy***

→ Initially the concept hierarchy was "day < month < quarter < year."

→ On drilling down, the time dimension is descended from the level of quarter to the level of month.

→When drill-down is performed, one or more dimensions from the data cube are added.

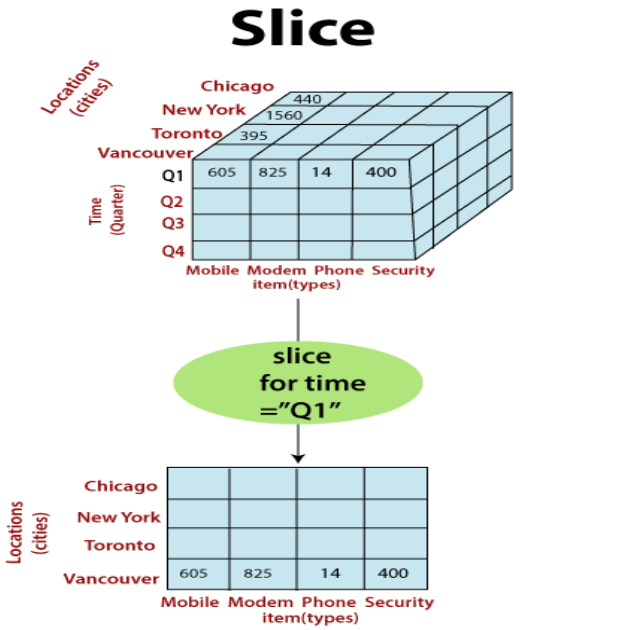


Slice and dice:

The slice operation selects one particular dimension from a given cube and provides a new sub-cube. Consider the following diagram that shows how slice works.

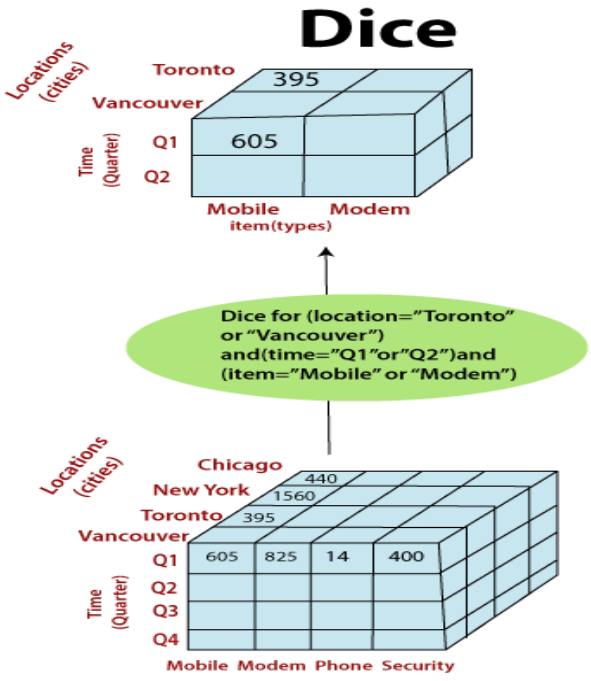
Here Slice is performed for the dimension "time" using the criterion time = "Q1".

It will form a new sub-cube by selecting one or more dimensions.



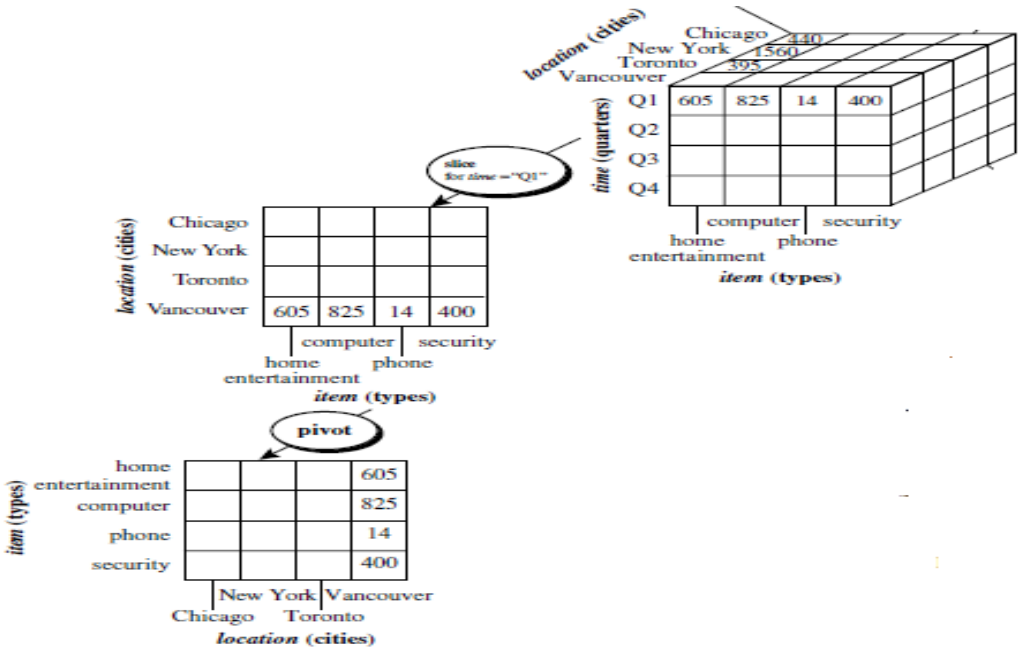
Dice selects two or more dimensions from a given cube and provides a new sub-cube.

Consider the following diagram that shows the dice operation. The dice operation on the cube based on the following selection criteria involves three dimension(location = "Toronto" or "Vancouver") (time = "Q1" or "Q2") (item =" Mobile" or "Modem")



Pivot (rotate):

- Visualization operation that rotates the data axes in view to provide an alternative data presentation


location (cities)
Chicago
New York
Toronto
Vancouver
440
1560
395
time (quarters)
Q1
Q2
Q3
Q4
605
825
14
400
computer
security
home entertainment
phone
item (types)
slice
for time = "Q1"
location (cities)
Chicago
New York
Toronto
Vancouver
605
825
14
400
computer
security
home entertainment
phone
item (types)
pivot
item (types)
home entertainment
computer
phone
security
605
825
14
400
New York
Vancouver
Chicago
Toronto
location (cities)